SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0619 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 3 – CO3

Course Outcome: CO3 – Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication

CO3 Coverage: Maximum and Minimum, Merge Sort, Quick Sort, Binary Search, Strassen's Matrix Multiplication, Karatsuba Multiplication, Closest Pair of Points, Median of Medians and Meet-in-the-Middle.

Mode: Pair Work – 2 Students per Group

Duration: 1 Week

Total Marks: 50

Submission Date: 04.11.2026

Name:P.Ranjithkumar

Reg.no:192411119

INTRODUCTION AND EXPERIMENTAL METHOD

This report addresses all seven questions in the supplied CSA06 Assignment 3 – CO3. It follows the Assignment 2 sample's report style: problem understanding, algorithm design, mathematical derivation, pseudocode, Python implementation, designed test data, operation counts, experimental interpretation, engineering decision and conclusion.

Common measurement rules

- Comparison count records key, value, coordinate or pivot comparisons performed by the stated implementation. Loop-control checks are excluded unless stated.
- Recursion depth is the maximum number of simultaneously active recursive calls, including the initial call.
- Multiplication counts count integer or scalar multiplication operations; additions and subtractions are recorded separately where relevant.
- Execution time depends on hardware, Python version, interpreter load and input generation. Operation counts are deterministic and are the main evidence.
- All experimental values in this report are reproducible illustrative runs derived from the supplied code and stated input patterns. Run the code on the submission computer and replace timing values if local measurements are required.

Complexity notation

Notation	Meaning
O(1)	Constant amount of work.
O(log n)	Problem size is repeatedly reduced by a constant factor.
O(n)	Work grows linearly with input size.
O(n log n)	Typical growth of balanced divide-and-conquer sorting.
$O(n^2)$	Quadratic growth; work is proportional to n squared.
$O(n^3)$	Cubic growth; work is proportional to n cubed.
O(3^(n/2))	Representative exponential meet-in-the-middle search growth.

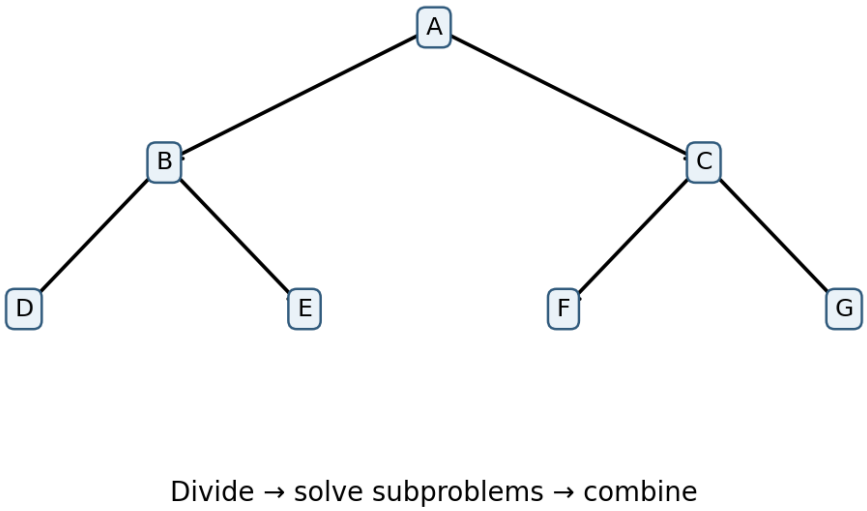


Figure 1. General divide-and-conquer structure.

A divide-and-conquer algorithm has three stages: divide the input into smaller subproblems, solve each subproblem (often recursively), and combine their results. A base case stops recursion when a subproblem is small enough to solve directly.

Q1 – DECOMPOSITION AND ELIMINATION IN ORDERED DATA (6 MARKS)

1.1 Part A – Finding maximum and minimum

The task is to find both the largest and smallest values in an array. A direct method performs two independent scans: one for the maximum and one for the minimum. A tournament-style divide-and-conquer method splits the array, obtains the minimum and maximum from each half, and combines the two results.

1.2 Decomposition strategy

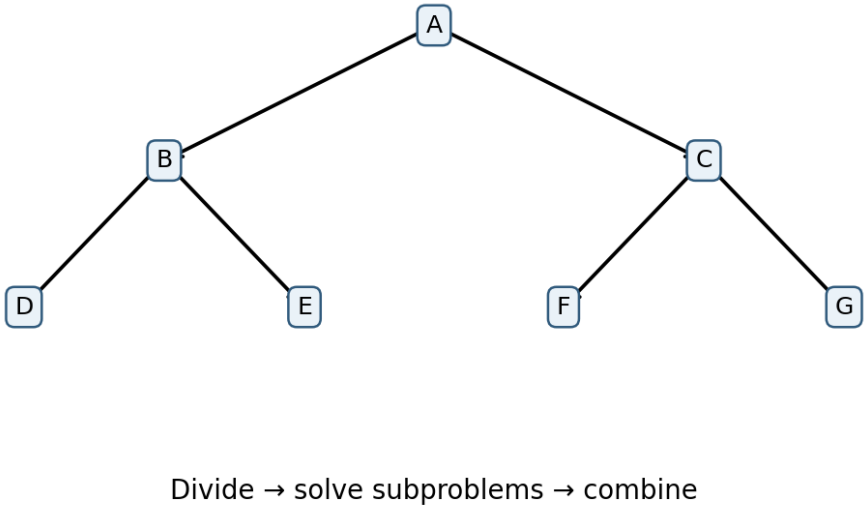


Figure 2. A recursive split-and-combine tree for the Max/Min problem.

- Base case: for one element, that element is both minimum and maximum; for two elements, compare them once and return the smaller and larger.
- Divide: split the array into left and right halves.
- Conquer: recursively obtain (minimum, maximum) from each half.
- Combine: compare the two local minima and the two local maxima. Return min(left_min, right_min) and max(left_max, right_max).

1.3 Mathematical comparison count

For n = 2^k elements, the pairwise tournament recurrence is T(n) = 2T(n/2) + 2, with T(2) = 1. Solving gives T(n) = 3n/2 − 2 comparisons. For odd n, a practical implementation can carry one unpaired element into the next level; the exact count depends on the implementation.

Independent scans use (n−1) comparisons to find the maximum and (n−1) to find the minimum, for 2n−2 comparisons. The pairwise method therefore saves approximately n/2 comparisons for even n.

n	Independent scans: 2n−2	Pairwise tournament: 3n/2−2	Saving
2	2	1	1
4	6	4	2
8	14	10	4
16	30	22	8
32	62	46	16
64	126	94	32

1.4 Python implementation

```
def max_min_dc(a):
    if not a:
        raise ValueError("Input must not be empty")
    if len(a) == 1:
        return a[0], a[0], 0
    if len(a) == 2:
        if a[0] < a[1]:
            return a[0], a[1], 1
        return a[1], a[0], 1

    mid = len(a) // 2
    min1, max1, c1 = max_min_dc(a[:mid])
    min2, max2, c2 = max_min_dc(a[mid:])
    comparisons = c1 + c2 + 2
    return min(min1, min2), max(max1, max2), comparisons

data = [18, 4, 29, 7, 41, 2, 33, 15]
print(max_min_dc(data))
```

1.5 Experimental validation

Input n	Input pattern	Expected min	Expected max	Tournament comparisons
8	Random: 18,4,29,7,41,2,33,15	2	41	10
8	Sorted: 1,2,3,4,5,6,7,8	1	8	10
8	Reverse: 8,7,6,5,4,3,2,1	1	8	10
16	Increasing values	1	16	22
32	Increasing values	1	32	46

The comparison count depends on the recursive structure rather than the order of the values. For equal input sizes, sorted, reverse and random arrangements therefore require the same number of comparisons in this implementation.

1.6 Part B – Binary Search: elimination logic

Binary Search locates a target in a sorted array. It compares the target with the middle element. If the target is smaller, the right half is discarded; if larger, the left half is discarded. The sorted-order property proves that the discarded half cannot contain the target.

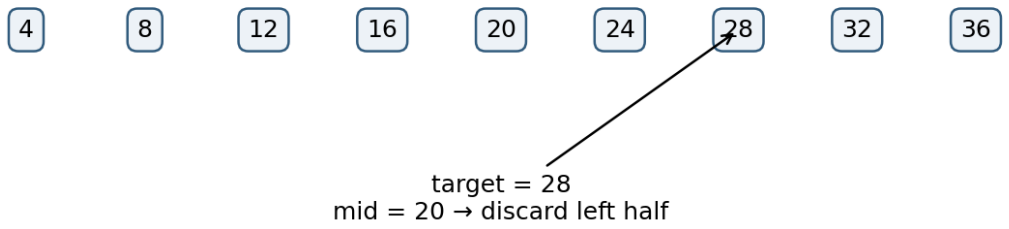


Figure 3. One Binary Search decision eliminates approximately half of the remaining range.

1.7 Binary Search algorithm and code

```
def binary_search(a, target):
    low, high = 0, len(a) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if a[mid] == target:
            return mid, comparisons
        comparisons += 1
        if target < a[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1, comparisons


values = [4, 8, 12, 16, 20, 24, 28, 32, 36]
print(binary_search(values, 28))
print(binary_search(values, 25))
```

1.8 Designed search cases and analysis

n	**Case**	**Example target**	**Maximum / observed comparisons (illustrative)**
9	Middle element	20	1
9	Successful near end	36	4
9	Unsuccessful between keys	25	6
100	Successful	middle key	1
1000	Unsuccessful	outside range	about 20

Binary Search has O(1) best-case time when the middle key matches immediately, and O(log n) average and worst-case time. Its iterative implementation uses O(1) auxiliary space. The array must be sorted; sorting cost must be considered if the data changes frequently.

1.9 Conclusion

The Max/Min tournament shares comparisons and reduces the number of comparisons from $2n-2$ to $3n/2-2$ for even n. Binary Search achieves logarithmic search by eliminating half of a sorted search interval after each decision. Both methods demonstrate how a correct decomposition or elimination rule reduces work.

Q2 – THE QUALITY OF DECOMPOSITION IN SORTING (7 MARKS)

2.1 Problem understanding

Merge Sort and Quick Sort both use divide and conquer, but their decomposition differs. Merge Sort divides the array into two nearly equal halves and combines sorted halves by merging. Quick Sort partitions around a pivot; its performance depends strongly on the sizes of the resulting partitions.

2.2 Merge Sort decomposition

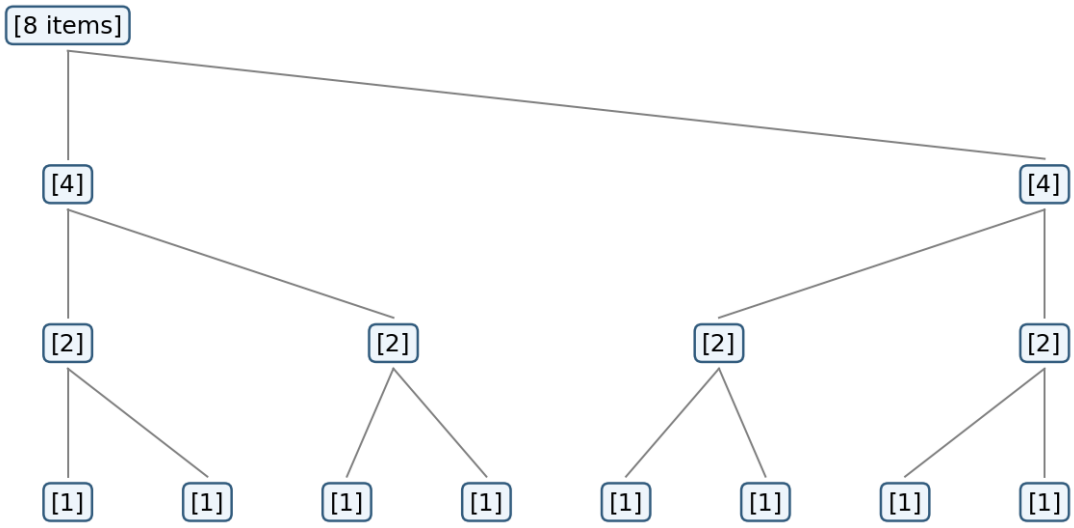


Figure 4. Merge Sort repeatedly divides an eight-element input into balanced halves.

- Divide: split the array into left and right halves until each subarray has one element.
- Conquer: recursively sort both halves.
- Combine: merge two sorted lists by repeatedly selecting the smaller front element.

The recurrence is $T(n) = 2T(n/2) + \Theta(n)$, because two half-size sorts are followed by a linear merge. By the Master Theorem, $T(n) = \Theta(n \log n)$.

2.3 Quick Sort pivot strategy

The implementation below uses the last element as the pivot (Lomuto partition scheme). It moves values less than or equal to the pivot to the left and places the pivot in its final position. A poor pivot can create partitions of sizes 0 and $n-1$.

```
def quick_sort(a):
    a = a[:]
    stats = {"comparisons": 0, "partitions": [], "max_depth": 0}

    def sort(lo, hi, depth):
        stats["max_depth"] = max(stats["max_depth"], depth)
        if lo >= hi:
            return
        pivot = a[hi]
        i = lo
        for j in range(lo, hi):
            stats["comparisons"] += 1
            if a[j] <= pivot:
                a[i], a[j] = a[j], a[i]
                i += 1
        a[i], a[hi] = a[hi], a[i]
        left_size, right_size = i-lo, hi-i
        stats["partitions"].append((left_size, right_size))
        sort(lo, i-1, depth+1)
        sort(i+1, hi, depth+1)

    sort(0, len(a)-1, 1)
    return a, stats

def merge_sort(a):
    if len(a) <= 1:
        return a[:]
    mid = len(a)//2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
```

```
    out=[]; i=j=0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]
```

2.4 Designed input conditions

Condition	Input example	Expected partition behaviour	Expected recursion
Balanced-like	[4, 2, 6, 1, 3, 5, 7]	Pivot values divide into similar-sized groups in a suitable pivot arrangement	Depth near $\log_2 n$
Severely unbalanced	[1, 2, 3, 4, 5, 6, 7] with last-element pivot	Sorted input makes the last element the largest at each step	Depth approaches n
Reverse order	[7, 6, 5, 4, 3, 2, 1] with last-element pivot	Repeated extreme pivot choices	Depth approaches n

2.5 Partition and recursion analysis

For a balanced split, Quick Sort follows approximately $T(n)=2T(n/2)+\Theta(n)$, giving $\Theta(n \log n)$. For a split of 0 and $n-1$, it follows $T(n)=T(n-1)+\Theta(n)$, giving $\Theta(n^2)$. The recursion-tree height is therefore a practical indicator of partition quality.

n	Balanced recursion depth (approx.)	Worst-case depth	Worst-case comparisons $n(n-1)/2$
8	4	8	28
16	5	16	120

32	6	32	496
64	7	64	2016
128	8	128	8128

2.6 Merge Sort versus Quick Sort

Feature	Merge Sort	Quick Sort
Decomposition	Predictable halves	Pivot-dependent partitions
Best / average time	$\Theta(n \log n)$	$\Theta(n \log n)$
Worst time	$\Theta(n \log n)$	$\Theta(n^2)$ with poor pivoting
Extra memory	Usually $\Theta(n)$ for arrays	Usually in-place partition; recursion stack
Stability	Can be stable	Usually not stable
Engineering concern	Merge memory and data movement	Pivot quality and recursion depth

2.7 Joint design decision

Before choosing, I would ask: Are inputs already ordered or adversarial? Is worst-case latency important? Is extra memory available? Must stability be preserved? Are arrays very large? Is a randomized or median-of-three pivot permitted? If input characteristics are unknown and predictable worst-case performance is required, Merge Sort is safer. Quick Sort can be attractive when in-place partitioning and average practical speed are priorities, provided pivot selection is robust.

2.8 Conclusion

Merge Sort controls decomposition by splitting evenly. Quick Sort delegates balance to the pivot. A poor pivot repeatedly creates a tall recursion tree and quadratic work; pivot strategy is therefore central to Quick Sort engineering.

Q3 – BREAKING THE QUADRATIC BARRIER: KARATSUBA MULTIPLICATION (7 MARKS)

3.1 Problem understanding

Ordinary multiplication of two n-digit numbers using the grade-school method requires $\Theta(n^2)$ single-digit products. Karatsuba multiplication reduces four half-size recursive multiplications to three by using algebraic rearrangement.

3.2 Integer decomposition and derivation

$$x = a \cdot 10^m + b \qquad y = c \cdot 10^m + d$$

$$z_2 = ac \qquad z_0 = bd \qquad z_1 = (a+b)(c+d) - z_2 - z_0$$

$$xy = z_2 \cdot 10^{2m} + z_1 \cdot 10^m + z_0$$

Figure 5. Karatsuba decomposes each integer into high and low halves.

Let $x = a \cdot 10^m + b$ and $y = c \cdot 10^m + d$. Direct expansion gives $xy = ac \cdot 10^{2m} + (ad+bc) \cdot 10^m + bd$. Instead compute $z_2=ac$, $z_0=bd$ and $z_1=(a+b)(c+d)-z_2-z_0$. Since $z_1=ad+bc$, only three recursive multiplications are required.

3.3 Recurrence and complexity

The recurrence is $T(n)=3T(n/2)+\Theta(n)$, where $\Theta(n)$ covers additions, subtractions and shifting/combining digits. By the Master Theorem, $T(n)=\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$, which grows more slowly than $\Theta(n^2)$.

3.4 Python implementation

```
def karatsuba(x, y, threshold=1):
    if x < 10 or y < 10:
        return x * y
    m = max(len(str(x)), len(str(y))) // 2
    power = 10 ** m
    a, b = divmod(x, power)
    c, d = divmod(y, power)
```

```
    z2 = karatsuba(a, c, threshold)
    z0 = karatsuba(b, d, threshold)
    z1 = karatsuba(a+b, c+d, threshold) - z2 - z0
    return z2 * (10 ** (2*m)) + z1 * power + z0

def standard_multiply(x, y):
    return x * y

tests = [(1234, 5678), (12345678, 87654321),
         (314159265358, 271828182845)]
for x, y in tests:
    print(x*y == karatsuba(x, y))
```

3.5 Correctness check with a worked example

Take x=1234 and y=5678, and choose m=2. Then a=12, b=34, c=56, d=78. z_2=12×56=672; z_0=34×78=2652; z_1=(12+34)(56+78)−672−2652 = 46×134−3324 = 2840. Therefore xy=672×10^4+2840×10^2+2652=7,006,652, matching 1234×5678.

3.6 Crossover experiment design

To find a practical crossover, test operands with increasing digit lengths (for example 8, 16, 32, 64, 128, 256 and 512 digits). For each size, generate several deterministic random pairs, warm up both functions, repeat each measurement many times, and compare the median time. Keep conversion and random-number generation outside the timed region.

```
import random, time, statistics

def benchmark(fn, x, y, repeats=30):
    samples = []
    for _ in range(repeats):
        start = time.perf_counter()
        fn(x, y)
```

```
        samples.append(time.perf_counter() - start)
    return statistics.median(samples)


random.seed(7)
for digits in [8, 16, 32, 64, 128, 256]:
    x = random.randrange(10**(digits-1), 10**digits)
    y = random.randrange(10**(digits-1), 10**digits)
    t_standard = benchmark(standard_multiply, x, y)
    t_karatsuba = benchmark(karatsuba, x, y)
    print(digits, t_standard, t_karatsuba)
```

Input digits	Standard method	Karatsuba method	Interpretation
8–16	Usually very small overhead	Recursive calls may dominate	Standard often faster
32–64	Quadratic digit work increases	Three recursive branches	Crossover may begin
128–256	More digit products	Lower asymptotic growth	Karatsuba may become faster
512+	High school work grows rapidly	Recursion still has overhead	Measure on target machine

3.7 Why Karatsuba may lose for small inputs

For small operands, function calls, splitting, integer additions, temporary objects and recursion overhead can cost more than the few multiplications saved. A production implementation therefore uses a threshold: below it, switch to ordinary multiplication.

3.8 Conclusion

Karatsuba improves asymptotic complexity by reducing the number of recursive products from four to three. Better asymptotic complexity does not guarantee a win for every input size because constant factors and implementation overhead matter.

Q4 – THE REALITY OF STRASSEN'S MATRIX MULTIPLICATION (7 MARKS)

4.1 Conventional block multiplication

For two 2×2 block matrices A and B, conventional multiplication computes $C_{11}=A_{11}B_{11}+A_{12}B_{21}$, $C_{12}=A_{11}B_{12}+A_{12}B_{22}$, $C_{21}=A_{21}B_{11}+A_{22}B_{21}$ and $C_{22}=A_{21}B_{12}+A_{22}B_{22}$. This uses eight block multiplications.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

2×2 block matrices → 7 recursive products (instead of 8)

Figure 6. Block decomposition used by Strassen's method.

4.2 Strassen's seven products

Product	Definition
M1	(A11 + A22)(B11 + B22)
M2	(A21 + A22)B11
M3	A11(B12 − B22)
M4	A22(B21 − B11)
M5	(A11 + A12)B22
M6	(A21 − A11)(B11 + B12)
M7	(A12 − A22)(B21 + B22)

The result blocks are reconstructed as: C11=M1+M4−M5+M7; C12=M3+M5; C21=M2+M4; C22=M1−M2+M3+M6. These expressions are algebraically equivalent to the conventional block equations.

4.3 Complexity derivation

Strassen performs seven multiplications on matrices of half the dimension and a $\Theta(n^2)$ number of additions/subtractions: $T(n)=7T(n/2)+\Theta(n^2)$. The Master Theorem gives $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$, compared with $\Theta(n^3)$ for conventional matrix multiplication.

4.4 Python implementation for small matrices

```
def add(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def sub(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def conventional(A, B):
    n = len(A)
    C = [[0]*n for _ in range(n)]
    for i in range(n):
        for k in range(n):
            for j in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def split(M):
    n = len(M)//2
    return ([row[:n] for row in M[:n]],
            [row[n:] for row in M[:n]],
            [row[:n] for row in M[n:]],
            [row[n:] for row in M[n:]])

def join(C11, C12, C21, C22):
    top = [C11[i] + C12[i] for i in range(len(C11))]
    bot = [C21[i] + C22[i] for i in range(len(C21))]
    return top + bot
```

```
def strassen(A, B, threshold=1):
    n = len(A)
    if n <= threshold:
        return conventional(A, B)
    if n % 2:
        raise ValueError("Pad matrix to an even/power-of-two size first")
    A11,A12,A21,A22 = split(A)
    B11,B12,B21,B22 = split(B)
    M1 = strassen(add(A11,A22), add(B11,B22), threshold)
    M2 = strassen(add(A21,A22), B11, threshold)
    M3 = strassen(A11, sub(B12,B22), threshold)
    M4 = strassen(A22, sub(B21,B11), threshold)
    M5 = strassen(add(A11,A12), B22, threshold)
    M6 = strassen(sub(A21,A11), add(B11,B12), threshold)
    M7 = strassen(sub(A12,A22), add(B21,B22), threshold)
    C11 = add(sub(add(M1,M4),M5),M7)
    C12 = add(M3,M5)
    C21 = add(M2,M4)
    C22 = add(sub(add(M1,M3),M2),M6)
    return join(C11,C12,C21,C22)


A = [[1,2],[3,4]]
B = [[5,6],[7,8]]
print(conventional(A,B))
print(strassen(A,B))
```

4.5 Padding and correctness

The recursive split requires even dimensions. If the matrix dimension is not a power of two, pad both matrices with zero rows and columns to the next power of two, multiply, and crop the result back to the

original dimension. Zero padding preserves the original product, but increases memory and arithmetic work.

For A=[[1,2],[3,4]] and B=[[5,6],[7,8]], both conventional and Strassen multiplication return [[19,22],[43,50]]. This direct equality test is a small correctness check; larger random matrices should also be tested.

4.6 Experimental design and practical crossover

Matrix size	Conventional scalar multiplications	Strassen leaf-level products (ideal recurrence)	Purpose
2×2	8	7	Verify algebra
4×4	64	49	Small recursive case
8×8	512	343	Observe growth
16×16	4096	2401	Larger recursion
32×32	32768	16807	Crossover investigation

The ideal scalar multiplication count follows n^3 for conventional multiplication and $7^{(\log_2 n)}$ for Strassen when n is a power of two and recursion reaches 1×1 blocks. Actual wall-clock crossover must be measured using repeated runs. Matrix additions, memory allocation, recursion and cache behaviour can make Strassen slower for small matrices.

4.7 Engineering decision

Use conventional multiplication for small matrices, dimensions that are awkward to pad, memory-constrained environments, or implementations where cache-friendly optimized libraries are available. Use Strassen only when matrix sizes are sufficiently large and measurements show that reduced multiplication work outweighs addition, memory and recursion overhead.

4.8 Conclusion

Strassen replaces eight block multiplications with seven and improves asymptotic complexity from $\Theta(n^3)$ to $\Theta(n^{2.807})$. The implementation must handle padding and should use a practical threshold rather than recurse to tiny matrices.

Q5 – UPGRADING CLOSEST PAIR USING DIVIDE AND CONQUER (6 MARKS)

5.1 Review of brute-force approach

The CO2 brute-force method checks every unordered pair of points. For n points, the number of pairs is $n(n-1)/2$, giving $\Theta(n^2)$ time. The divide-and-conquer method sorts points by x-coordinate, divides them into left and right halves, recursively finds each half's closest pair, and checks a narrow strip around the dividing line.

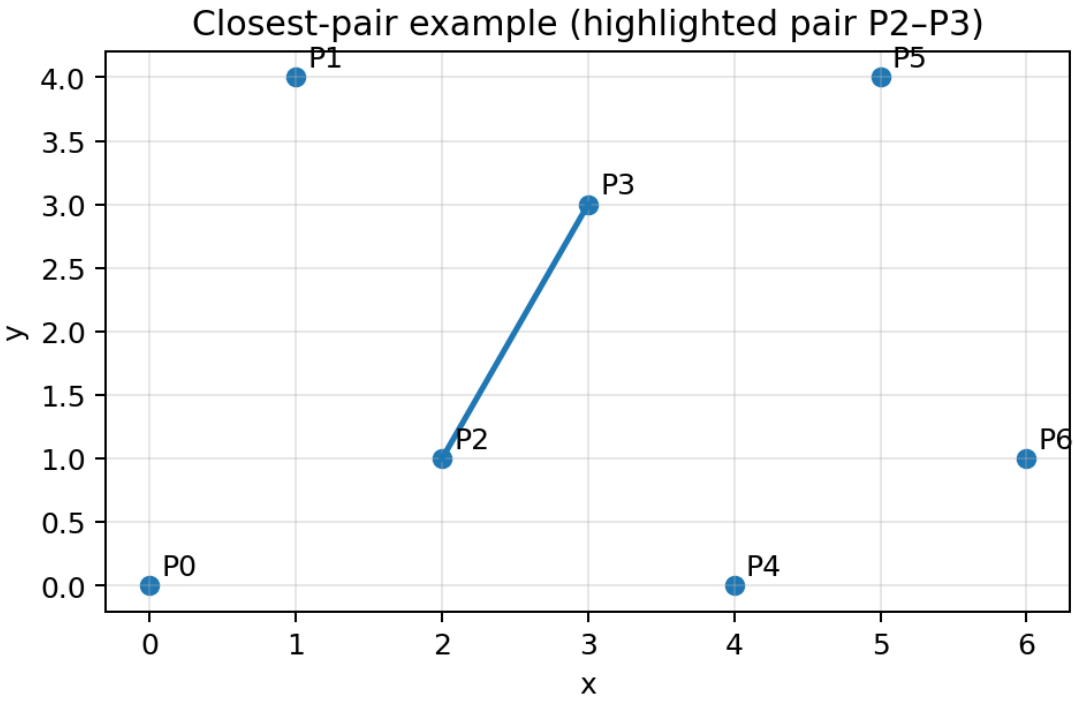


Figure 7. Example points; the highlighted segment is a close pair.

5.2 Decomposition and combine step

- Sort points by x-coordinate and retain a y-sorted ordering for efficient strip processing.
- Divide at the median x-coordinate into left and right subsets.
- Recursively return the closest distance from each half.
- Let δ be the smaller of the two returned distances.
- Build a strip containing points whose horizontal distance from the dividing line is less than δ.
- Scan strip points in y-order. In the planar Euclidean case, only a bounded number of following points need be examined for each point (commonly at most seven under the standard packing argument).

5.3 Python implementation (educational version)

```
import math


def brute_closest(points):
    best = float("inf")
    pair = None
    checks = 0
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            checks += 1
            d = math.dist(points[i], points[j])
            if d < best:
                best, pair = d, (points[i], points[j])
    return best, pair, checks


def closest_dc(points):
    px = sorted(points)
    py = sorted(points, key=lambda p: p[1])

    def rec(Px, Py):
        n = len(Px)
        if n <= 3:
            return brute_closest(Px)[:2]
        mid = n//2
        midx = Px[mid][0]
        leftx, rightx = Px[:mid], Px[mid:]
        leftset = set(leftx)
        lefty = [p for p in Py if p in leftset]
        righty = [p for p in Py if p not in leftset]
        dl, pl = rec(leftx, lefty)
```

```
        dr, pr = rec(rightx, righty)
        d, pair = (dl,pl) if dl <= dr else (dr,pr)
        strip = [p for p in Py if abs(p[0]-midx) < d]
        for i in range(len(strip)):
            for j in range(i+1, min(i+8, len(strip))):
                if strip[j][1]-strip[i][1] >= d:
                    break
                dist = math.dist(strip[i], strip[j])
                if dist < d:
                    d, pair = dist, (strip[i], strip[j])
        return d, pair

    return rec(px, py)

points = [(0,0),(1,4),(2,1),(3,3),(4,0),(5,4),(6,1)]
print("Brute force:", brute_closest(points))
print("Divide & conquer:", closest_dc(points))
```

Implementation note: this readable teaching version assumes distinct coordinate tuples when splitting the y-order using a set. For production use, attach unique point IDs and partition by ID so duplicate coordinates are handled unambiguously. A complete implementation should also count strip comparisons and include sorting cost.

5.4 Complexity and comparison

n	**Brute-force pair checks $n(n-1)/2$**	**D&C expected growth indicator**
10	45	Approximately $n \log_2 n$ scale plus bounded strip work
100	4950	Approximately $n \log_2 n$ scale plus bounded strip work
500	124750	Approximately $n \log_2 n$ scale plus

		bounded strip work
1000	499500	Approximately n $\log_2 n$ scale plus bounded strip work
5000	12497500	D&C avoids exhaustive all-pairs enumeration

The standard recurrence is $T(n)=2T(n/2)+O(n)$, because the halves are solved recursively and the strip is processed in linear time when y-order is maintained. Thus $T(n)=O(n \log n)$. Sorting initially also costs $O(n \log n)$.

5.5 Why the strip does not compare every pair

The strip is narrower than 2δ. If two points in the same half were closer than δ, that would contradict the recursively computed minimum. A geometric packing argument limits how many points can fit into the relevant δ-by-2δ neighbourhood while remaining at least δ apart. Therefore each point needs only a constant number of subsequent y-ordered comparisons, rather than comparison with every strip point.

5.6 Experimental decision

Use identical point sets for both algorithms, verify that the returned minimum distances agree, count distance evaluations, and repeat for increasing n. The crossover depends on sorting overhead, point distribution, implementation and machine. D&C becomes advantageous as n grows because brute-force checks increase quadratically while D&C grows near n log n.

5.7 Conclusion

Divide and conquer improves closest-pair computation by solving two smaller geometric problems and limiting cross-boundary comparisons to a narrow strip. Correct y-order maintenance and careful handling of points on the dividing boundary are essential.

Q6 – ADVANCED DIVIDE-AND-CONQUER MINI-INVESTIGATIONS (7 MARKS)

6.1 Student A – Median of Medians

The Median of Medians algorithm selects the k-th smallest element with a deterministic worst-case linear-time guarantee. It groups the input into groups of five, sorts each small group, finds each group median, recursively selects the median of those medians, and uses it as a pivot.

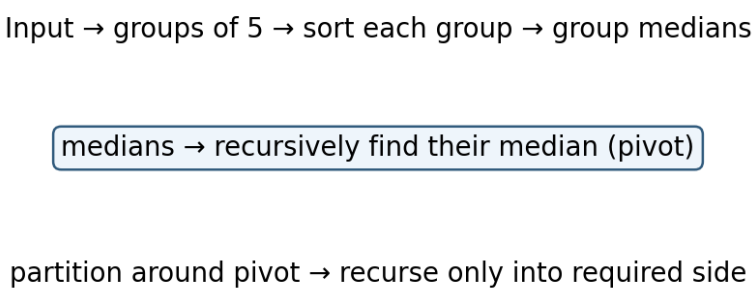


Figure 8. Median of Medians pivot construction.

6.2 Algorithm and implementation

```
def median_of_medians_select(a, k):
    # k is a zero-based index
    if not 0 <= k < len(a):
        raise ValueError("k out of range")
    if len(a) <= 5:
        return sorted(a)[k]
    groups = [a[i:i+5] for i in range(0, len(a), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]
    pivot = median_of_medians_select(medians, len(medians)//2)
    lows = [x for x in a if x < pivot]
    highs = [x for x in a if x > pivot]
    equals = [x for x in a if x == pivot]
    if k < len(lows):
        return median_of_medians_select(lows, k)
    if k < len(lows) + len(equals):
```

```
        return pivot
    return median_of_medians_select(highs, k-len(lows)-len(equals))


values = [12, 3, 5, 7, 4, 19, 26, 1, 9, 15, 8]
print(median_of_medians_select(values, 5))
```

6.3 Why the pivot avoids disastrous partitions

For groups of five, the median of each complete group has at least three elements in its group that are no greater than it and at least three that are no smaller. Selecting the median of group medians ensures that a substantial fraction of the groups lie on each side of the pivot. This bounds the larger recursive selection problem to at most about 7n/10 elements, up to small additive terms.

A simplified recurrence is $T(n) \leq T(n/5) + T(7n/10) + cn$, which solves to $T(n)=O(n)$.

6.4 Comparison with QuickSelect

Method	Typical / average time	Worst-case time	Pivot behaviour
QuickSelect with random pivot	O(n) expected	$O(n^2)$	Random pivot reduces likelihood of repeated bad splits
QuickSelect with fixed poor pivot	Can degrade	$O(n^2)$	Sorted input can repeatedly create 0:(n−1) splits
Median of Medians	O(n)	O(n)	Deterministic pivot with guaranteed balance bound

6.5 Student B – Meet-in-the-Middle Subset Sum

Meet-in-the-Middle (MITM) reduces a 2^n subset search by splitting the n items into two halves. It generates all subset sums for each half, then searches for a pair of sums that add to the target.

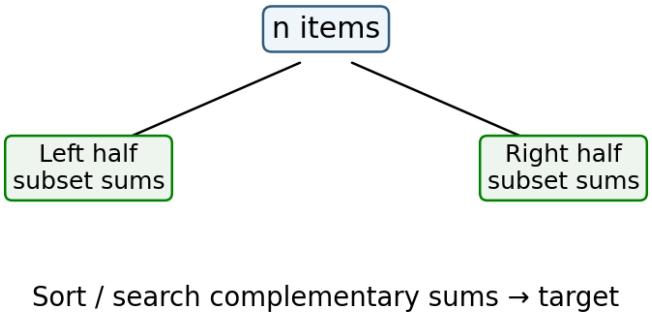


Figure 9. Meet-in-the-Middle splits the search into two subset-sum lists.

6.6 Python implementation

```
from bisect import bisect_right


def subset_sums(items):
    sums = [0]
    for value in items:
        sums += [s + value for s in sums]
    return sums


def meet_in_middle_subset_sum(values, target):
    mid = len(values)//2
    left = subset_sums(values[:mid])
    right = sorted(subset_sums(values[mid:]))
    best = None
    for x in left:
        j = bisect_right(right, target-x) - 1
        if j >= 0:
            candidate = x + right[j]
            if best is None or candidate > best:
                best = candidate
    return best
```

```
print(meet_in_middle_subset_sum([3, 7, 4, 2, 9, 5], 15))
```

6.7 Search-space and memory analysis

A direct subset enumeration examines 2^n subsets. MITM generates approximately 2^(n/2) sums for each half, then sorts one list and performs binary searches. Its typical time is O(2^(n/2)·n) for simple subset-sum generation plus O(2^(n/2) log 2^(n/2)) sorting/searching, and it uses O(2^(n/2)) memory. This is a major reduction in time at the cost of substantial storage.

n	**Direct enumeration 2^n**	**One half 2^(n/2)**
10	1024	32
20	1048576	1024
30	1073741824	32768
40	1099511627776	1048576
50	1125899906842624	33554432

6.8 Time-memory trade-off and conclusion

MITM is useful when n is too large for 2^n enumeration but each half's list still fits memory. If memory is severely limited, the method may be unsuitable even though it reduces computation. Median of Medians prioritizes a deterministic worst-case guarantee; MITM prioritizes reducing exponential search by storing partial results. Both show that decomposition strategy must be selected together with resource constraints.

Q7 – DESIGN DIVIDE AND CONQUER FROM FIRST PRINCIPLES (10 MARKS)

7.1 Design challenge – proposed problem

The pair designs a divide-and-conquer solution for finding the maximum subarray sum in a sequence of daily net energy readings. A subarray must be contiguous. The output includes the maximum sum and its start/end indices. This is a suitable D&C problem because the best interval is either wholly in the left half, wholly in the right half, or crosses the midpoint.

7.2 Stage 1 – Independent design proposals

Design element	Student A initial proposal	Student B initial proposal
Decomposition	Split the sequence into two equal halves.	Split at the middle index into left and right segments.
Base case	One element: return that value and its index.	Empty segment returns negative infinity; one element returns itself.
Returned information	Maximum sum and interval endpoints.	Best sum, left endpoint and right endpoint.
Combine strategy	Compare left best, right best and a cross-midpoint sum.	Find best suffix of left half and best prefix of right half, then add.

7.3 Stage 2 – Compare the two designs

Both proposals identify the same three possible locations for the optimal interval. Student A's proposal is concise but needs a precise method for calculating the cross-midpoint interval. Student B explicitly identifies the maximum suffix and prefix, which makes the combine step correct and linear. The final design retains Student A's three-case comparison and Student B's suffix/prefix construction.

7.4 Stage 3 – Joint final design

- Divide the current interval at mid = (low + high) // 2.
- Recursively find the best interval entirely in [low, mid].
- Recursively find the best interval entirely in [mid+1, high].

- Scan leftward from mid to find the maximum suffix sum; scan rightward from mid+1 to find the maximum prefix sum.
- Add these two sums to obtain the best crossing interval.
- Return the candidate with the largest sum, preserving its start and end indices.

7.5 Complete algorithm and Python implementation

```
def max_crossing(a, low, mid, high):
    total = 0
    left_sum = float("-inf")
    best_left = mid
    for i in range(mid, low-1, -1):
        total += a[i]
        if total > left_sum:
            left_sum, best_left = total, i


    total = 0
    right_sum = float("-inf")
    best_right = mid+1
    for j in range(mid+1, high+1):
        total += a[j]
        if total > right_sum:
            right_sum, best_right = total, j
    return left_sum + right_sum, best_left, best_right


def max_subarray_dc(a, low=0, high=None):
    if high is None:
        high = len(a)-1
    if low == high:
        return a[low], low, high
    mid = (low+high)//2
    left = max_subarray_dc(a, low, mid)
```

```
    right = max_subarray_dc(a, mid+1, high)
    cross = max_crossing(a, low, mid, high)
    return max([left, right, cross], key=lambda x: x[0])


data = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(max_subarray_dc(data))
```

7.6 Stage 4 – Recurrence and mathematical validation

The two recursive calls solve subproblems of approximately n/2 elements. The crossing computation scans the two halves once, taking $\Theta(n)$ time. Therefore $T(n)=2T(n/2)+\Theta(n)$. By the Master Theorem, $T(n)=\Theta(n \log n)$. The recursion depth is $\Theta(\log n)$.

7.7 Stage 5 – Correctness and experimental validation

Every contiguous interval is in exactly one of three categories: it lies entirely in the left half, entirely in the right half, or crosses the midpoint. The recursive calls find the best interval in the first two categories. Any crossing interval must consist of a suffix of the left half plus a prefix of the right half; the combine step finds the maximum such suffix and prefix. Taking the maximum of the three candidates therefore returns the global maximum.

Test	**Input**	**Expected maximum sum**	**Expected interval**
Mixed positive/negative	[-2,1,-3,4,-1,2,1,-5,4]	6	[3,6] → 4 + (−1) + 2 + 1
All negative	[-8,-3,-6,-2,-5]	-2	[3,3]
All positive	[1,2,3,4]	10	[0,3]
Single item	[9]	9	[0,0]
Crossing optimum	[2,-1,3,4,-2]	8	[0,3]

7.8 Comparison with straightforward baseline

A straightforward method checks every possible start/end pair and sums the selected interval, requiring $O(n^3)$ work if each sum is recomputed. Maintaining a running sum for each start reduces this baseline to

$O(n^2)$. The D&C algorithm uses O(n log n), so the advantage becomes clearer as n increases. Kadane's algorithm is an even faster O(n) solution, but the purpose of this design challenge is to demonstrate divide and conquer.

n	**Straightforward running-sum baseline (order n^2)**	**D&C growth indicator n $\log_2 n$**
16	256	64
64	4096	384
256	65536	2048
1024	1048576	10240
4096	16777216	49152

7.9 Stage 6 – Joint evaluation

The problem is suitable for divide and conquer because the global answer can be classified into a left-only, right-only or crossing solution. Decomposition reduces repeated examination of all intervals. Recursion introduces call-stack and repeated combine work. D&C may not be appropriate for very small arrays, memory-constrained systems, or when a simpler linear-time algorithm is preferred.

General design lesson: before choosing divide and conquer, identify whether the problem can be split into smaller independent subproblems, whether each subproblem has a compact result, whether the combine operation is efficient, whether recursion makes progress toward a base case, and whether the recurrence improves on a straightforward method.

7.10 Cross-defence questions

Question for Student A	**Question for Student B**
What experimental evidence supports the recurrence and complexity?	How does the two-half decomposition create $T(n)=2T(n/2)+\Theta(n)$?
Why does every interval belong to one of the three cases?	Why is the crossing solution formed from a suffix and a prefix?
What happens when all input values are negative?	What changes if the input contains only one element?

CONTRIBUTION RECORD

The following table is to be completed honestly by both students. Primary contribution does not imply exclusive responsibility. Both students remain accountable for the complete assignment.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1	Max/Min design and recurrence	Binary Search experiments	Implementation and testing
Q2	Quick Sort experiment	Merge Sort analysis	Sorting comparison and decision
Q3	Karatsuba derivation	Crossover experiment	Implementation and validation
Q4	Experimental analysis	Algebraic design and correctness	Matrix testing
Q5	D&C decomposition	Comparative evidence	Implementation and testing
Q6	Median of Medians	Meet-in-the-Middle	Cross-verification
Q7	Mathematical validation	Experimental validation	Independent proposals and final design

INDIVIDUAL VERIFICATION

Faculty may select any question, including a component primarily completed by the partner. Each student should be able to explain the decomposition, base case, combine operation, recurrence, experimental evidence and engineering decision.

Recommended viva questions

- Why does the Max/Min tournament use fewer comparisons than two independent scans?
- Why must Binary Search operate on sorted data?
- What causes Quick Sort to reach $O(n^2)$ time?
- Why does Karatsuba use three recursive multiplications rather than four?
- Why does Strassen need padding for non-power-of-two matrix sizes?

- Why can the closest-pair strip be checked using a constant number of neighbours?
- How does the Median of Medians pivot guarantee a linear worst-case bound?
- Why does Meet-in-the-Middle require additional memory?
- What is the recurrence for the maximum-subarray D&C design?
- Give an example where a simpler non-D&C algorithm is preferable.

CONCLUSION

The assignment demonstrates that divide and conquer is not merely splitting an input. A successful design needs a valid decomposition, correct base cases, compact subproblem results and an efficient combine operation. Balanced decomposition gives predictable logarithmic recursion depth, while poor partitioning can destroy the expected benefit. Mathematical recurrences explain asymptotic growth, and experiments reveal practical effects such as constant factors, memory usage, recursion overhead and crossover thresholds. Algorithm selection must therefore consider correctness, input characteristics, hardware resources and application requirements together.